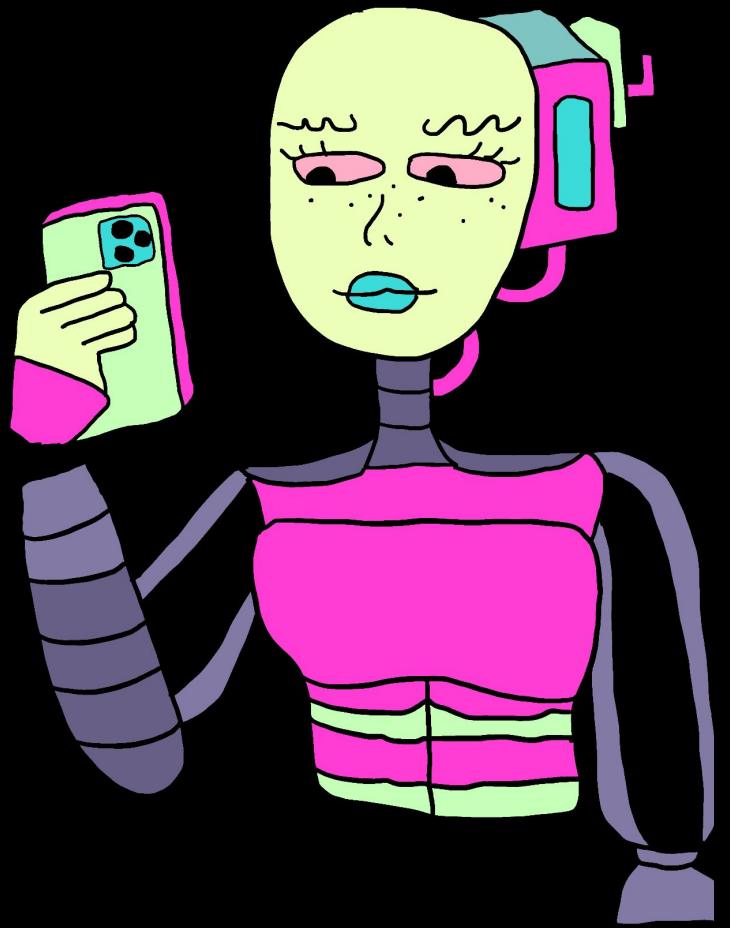
Da Gramática ao Algoritmo

Processamento de linguagem natural

Ana Laura - Python Brasil 2025



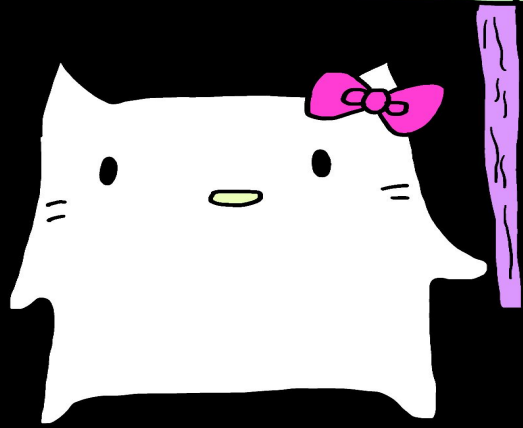
A Linguagem é uma das coisas mais legais e impressionante que os humanos fazem



Existe de várias formas, tamanhos. Pode ser falada ou escrita.



A IA tem sido amplamente usada para **processar dados numéricos e imagens**, mas hoje também entende e gera linguagem nossa principal forma de interação com computadores.



O que e **processamento de linguagem natural?**



- Compreensão de Linguagem Natural - NLU
- Geração de Linguagem Natural - NLG



O desafio: **entender o significado das palavras**

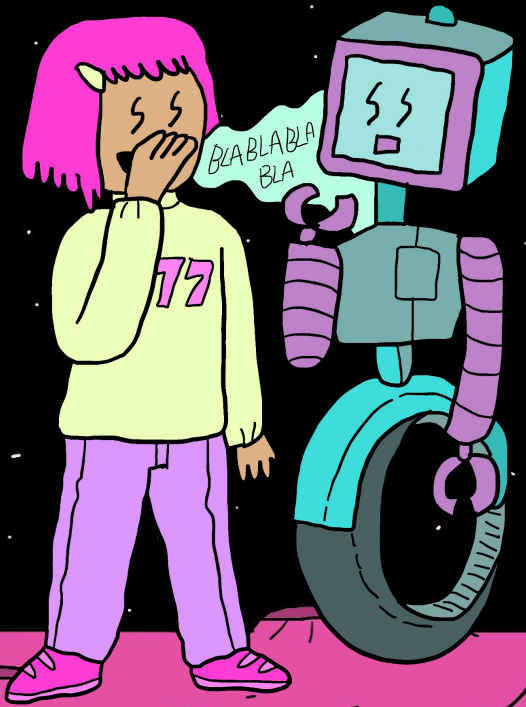
- As palavras não têm significado por si só nós é que atribuímos sentido a símbolos.

- E a linguagem é ambígua: o contexto muda tudo.



Em NLP (NLU ou NLG), **a máquina também precisa**:

- Aprender o significado das palavras
- Entender o contexto e nossos possíveis erros

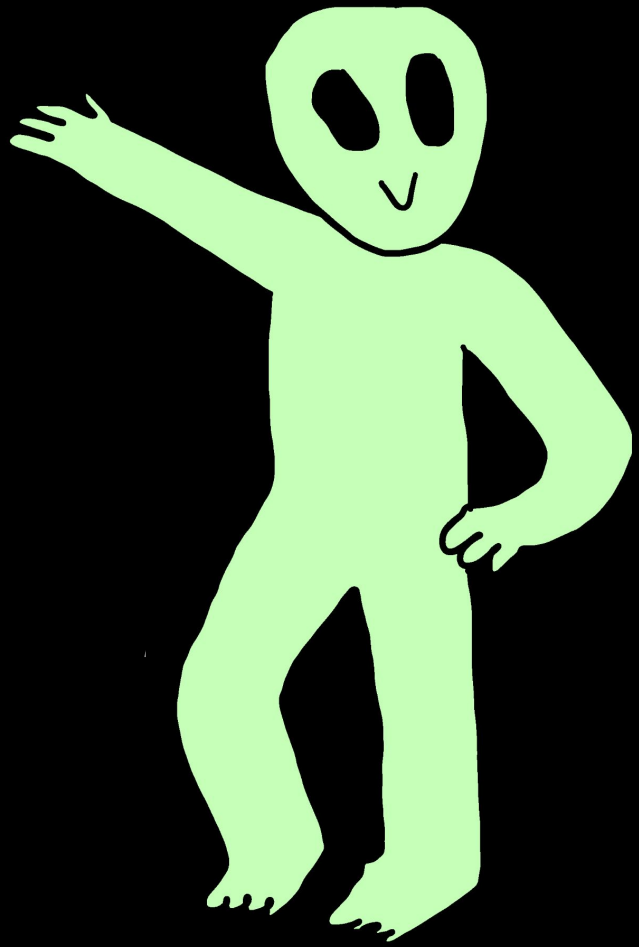


Primeiras Tentativas: O Processamento Simbólico

- As primeiras tentativas foram baseadas na gramática que aprendemos na escola.

Processamento Simbólico: Modelar a linguagem com regras formais.

- Cada palavra tem uma categoria (ex.: substantivo, verbo).
- As frases são formadas por regras de produção (ex.: sujeito + verbo + predicado).
- Permite construir árvores sintáticas.



```python
import spacy

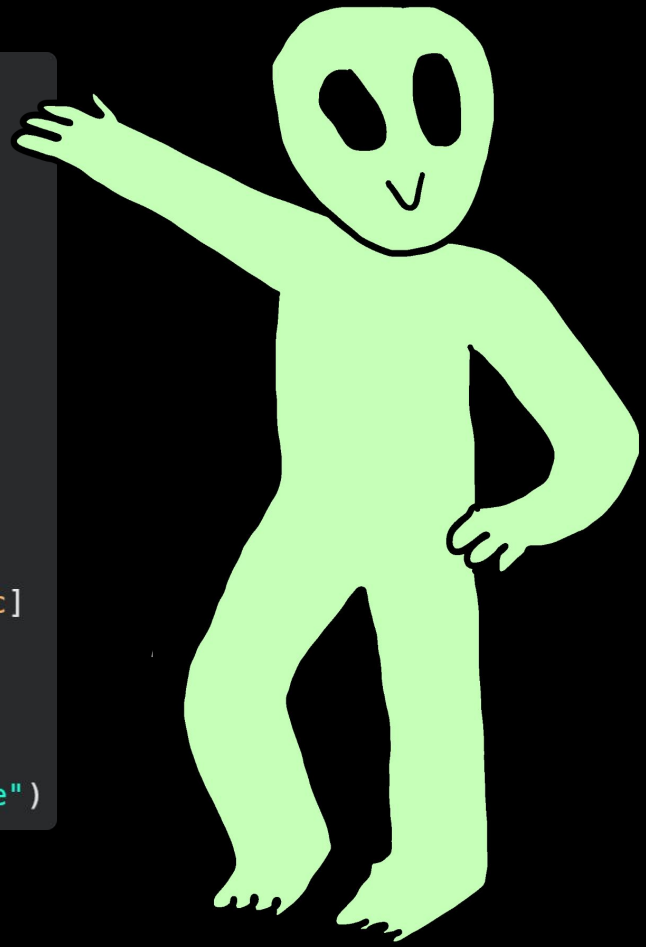
# Load the Portuguese spaCy model
nlp = spacy.load("pt_core_news_sm")

frase = "O gato pulou no sofá"

# Process the sentence with spaCy
doc = nlp(frase)

# Extract tokens and POS tags
tags_spacy = [(token.text, token.pos_) for token in doc]

print("POS tagging using spaCy:")
print(tags_spacy)
    sim = similaridade_morfologica(p1, p2)
    print(f"'{p1}' vs '{p2}': {sim:.2f} de similaridade")
```



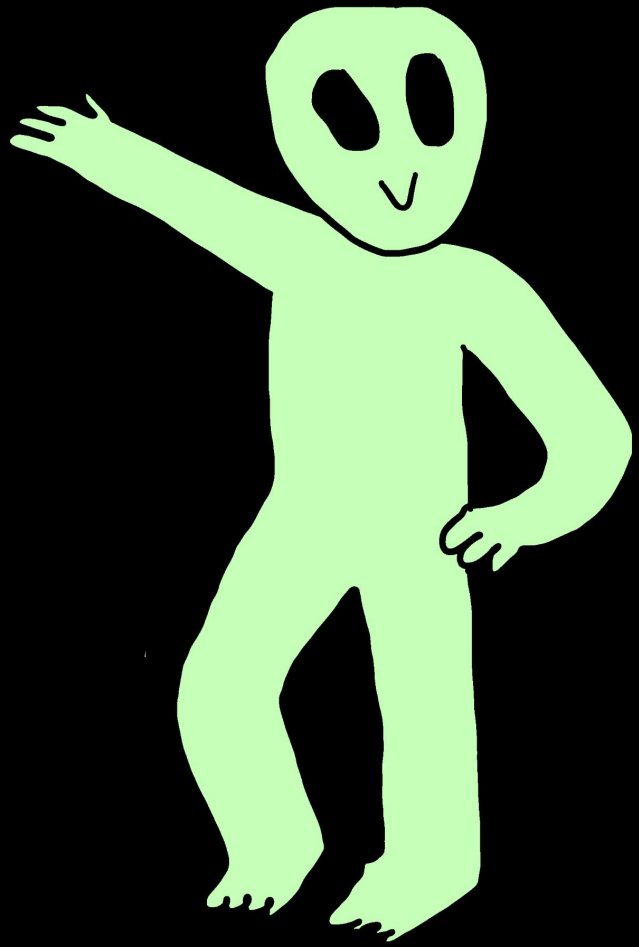
```
Palavra → Classe gramatical
------------------------------
O           → DET
gato        → NOUN
pulou       → VERB
no          → ADP
sofá        → NOUN
```



A Virada Estatística: Modelos Probabilísticos

Para lidar com a ambiguidade dos dados reais, surgiram os **Modelos Probabilísticos.**

- N-gramas: Contam sequências de palavras para prever a próxima.
- Cadeias de Markov: A probabilidade de uma palavra depende das anteriores.



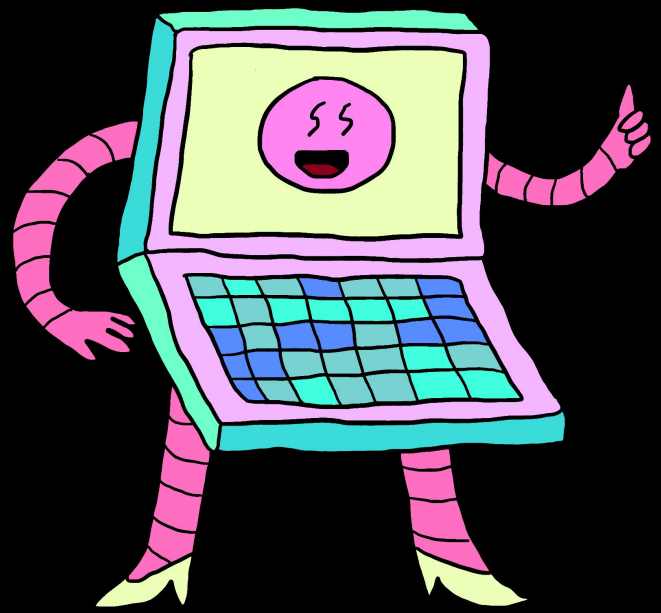
Morfologia e Semelhança Mais Simples

Podemos comparar palavras pelas letras que compartilham.

Funciona quando há morfologia: uma raiz comum que se modifica.

Exemplos:

- **nadar → nadando, nadador**
- **beber → bebendo, bebedor**
- **pensar → pensando, pensador**



Morfologia e Semelhança Mais Simples

```python
def similaridade_morfologica(palavra1, palavra2):
    # Método simples baseado em caracteres compartilhados
    chars1 = set(palavra1)
    chars2 = set(palavra2)
    similaridade = len(chars1.intersection(chars2)) / len(chars1.union(chars2))
    return similaridade

exemplos = [("nadar", "nadando"), ("beber", "bebedor"), ("pensar", "pensador")]
for p1, p2 in exemplos:
    sim = similaridade_morfologica(p1, p2)
    print(f"'{p1}' vs '{p2}': {sim:.2f} de similaridade")
```

Morfologia e Semelhança Mais Simples

```
1️⃣ MORFOLOGIA E SEMELHANÇA
------------------------------
'nadar' vs 'nadando': 0.60 de similaridade
'beber' vs 'bebedor': 0.60 de similaridade
'pensar' vs 'pensador': 0.75 de similaridade
```

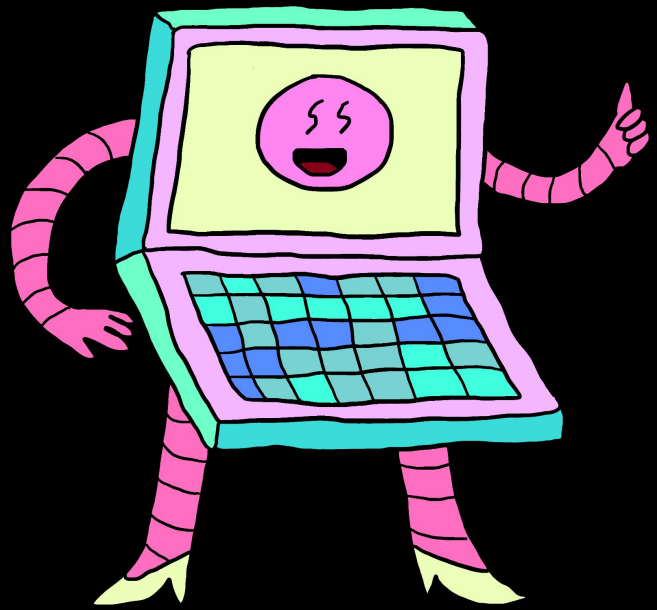
A Revolução do Significado: Semântica Distribucional

Palavras **com significados similares aparecem** frequentemente nas mesmas sentenças.
Baseia-se em insights da linguística:

"Você conhecerá uma palavra pela companhia que ela mantém." – John Firth

Para os computadores entenderem isso, precisamos traduzir em matemática.

O PLN une linguística, lógica e matemática para processar a linguagem.



Vetores de Contagem

Registrar **quantas vezes uma palavra aparece junto de outras palavras** selecionadas em um texto.

```python
from sklearn.feature_extraction.text import
CountVectorizer

corpus = [
    "O gato miou alto",
    "O cachorro latiu alto",
    "O gato dormiu"
]

vetorizador = CountVectorizer()
X = vetorizador.fit_transform(corpus)

print(vetorizador.get_feature_names_out())
print(X.toarray())
```

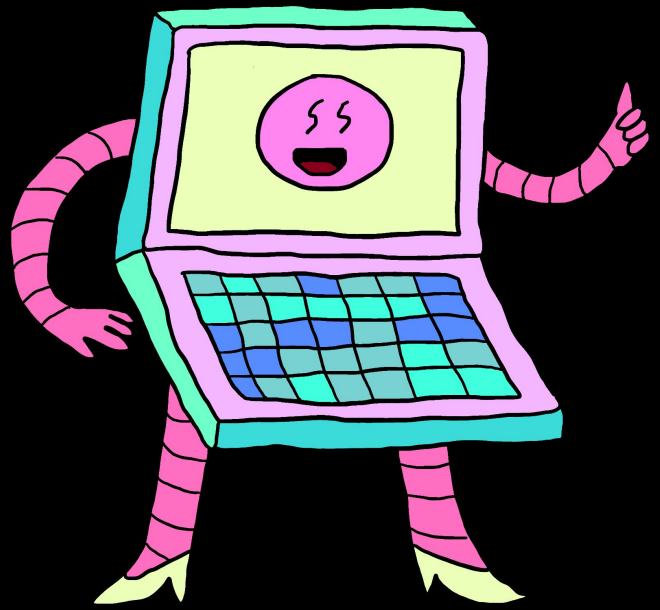


```
['alto' 'cachorro' 'dormiu' 'gato' 'latiu' 'miou']
[[1 0 0 1 0 1]
 [1 1 0 0 1 0]
 [0 0 1 1 0 0]]
```



Como funciona?

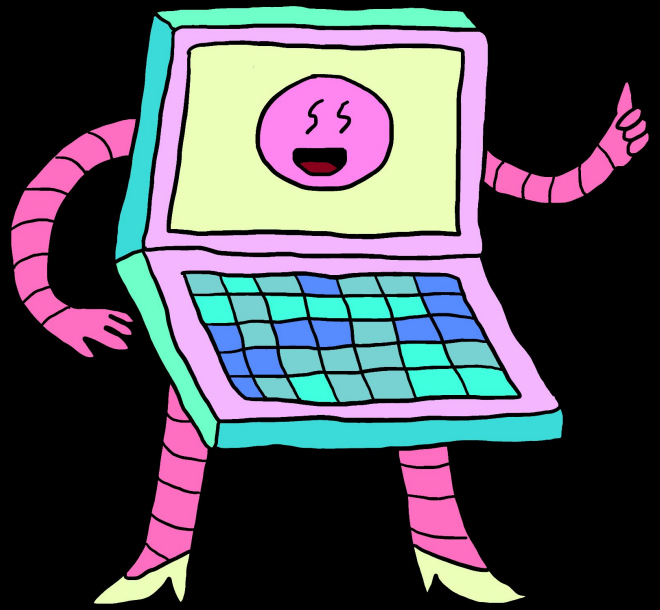
- Escolhe-se textos (ex.: Wikipédia, posts).
- Remove-se palavras comuns (stopwords).
- Conta-se frequência de palavras próximas.



Exemplos:

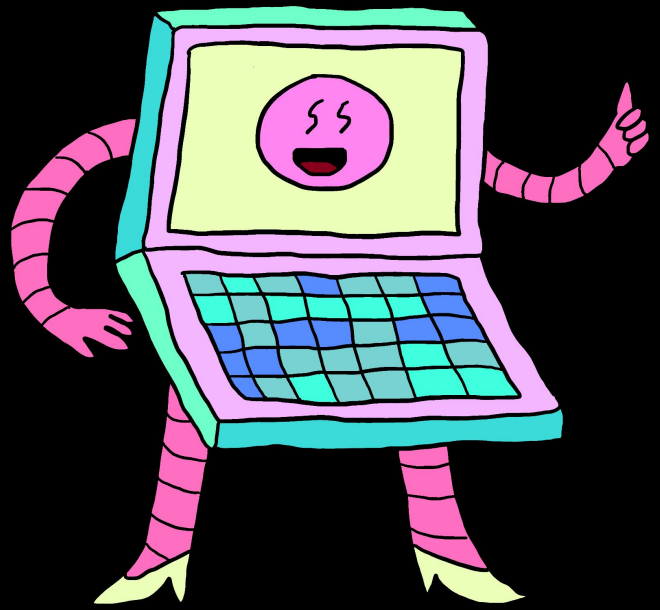
- Gato → aparece com "animal" e "miar".
- Felino → aparece com "animal" e "miar".
- Carro → aparece apenas com "carro".

★ **Palavras com vizinhos parecidos → significados próximos.**
★ **Palavras com contextos diferentes → significados distantes.**



De contagem a significado: embeddings

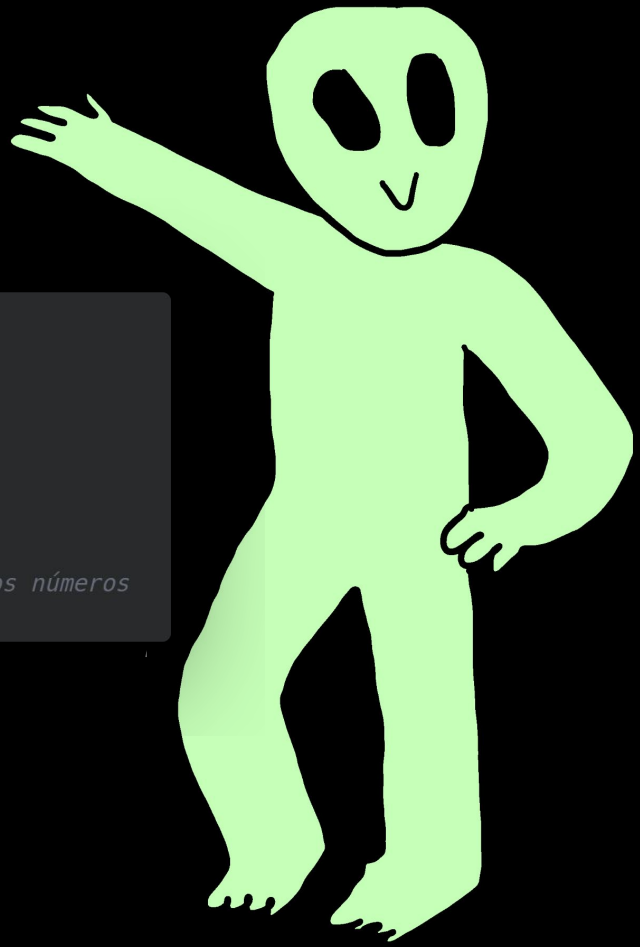
- ★ O computador aprende vetores que representam o contexto das palavras.
- ★ Palavras próximas em sentido ficam próximas no espaço vetorial.
- ★ Exemplo: “rei - homem + mulher ≈ rainha”.
- ★ o “significado” passa a ter forma numérica.



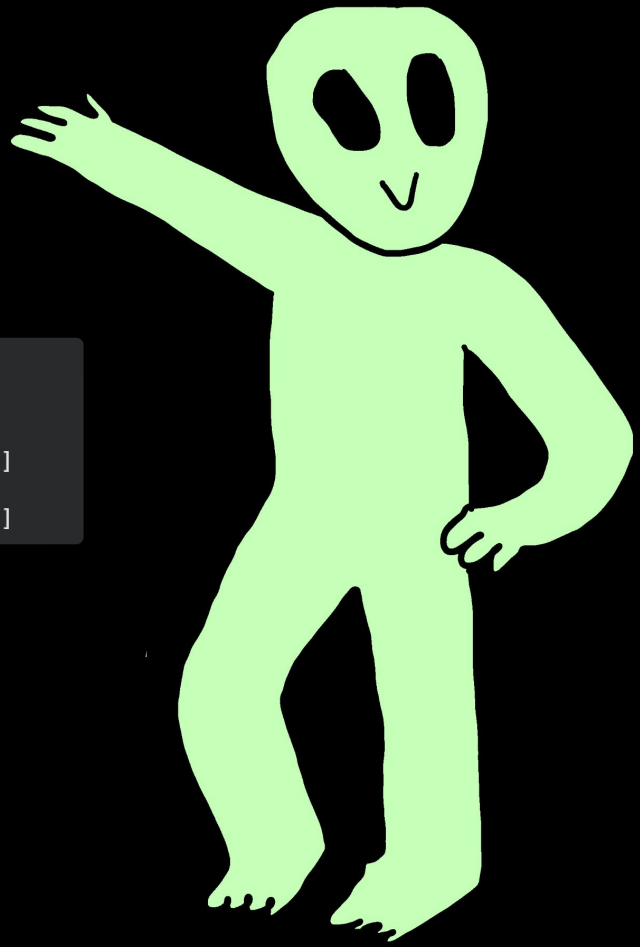
```python
import spacy
nlp = spacy.load("pt_core_news_sm")

palavras = ["rei", "rainha", "homem", "mulher"]

for p in palavras:
    print(p, "→", nlp(p).vector[:5])  # mostra só os 5 primeiros números
```

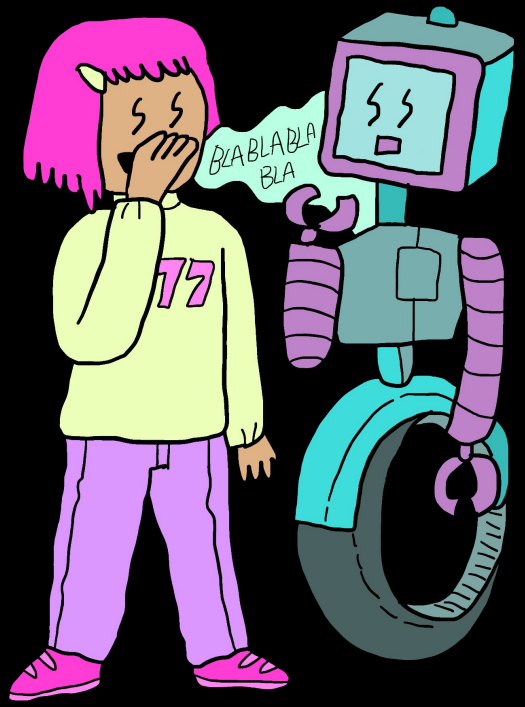


```
rei → [-2.3775558  0.791057   2.2338045  6.2460117 -1.5438154]
rainha → [-0.6937222   6.3993464   2.9622824   2.3494003  -0.24678737]
homem → [ 1.8478551  -1.4598114   0.25947765  2.9552598   2.2985017 ]
mulher → [-2.2727947  -2.052946   -1.126592    4.9124913  -0.15569168]
```



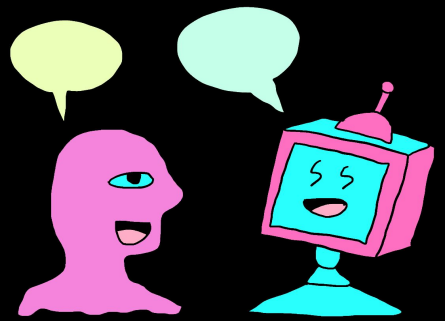
Modelagem de Linguagem – “Preencher a Lacuna”

Agora que temos representações numéricas, o modelo pode tentar prever o que vem a seguir.



Modelagem de Linguagem – “Preencher a Lacuna”

- Jogo de preencher a lacuna: “Queria algo de chocolate, poderia ser um ___.”
- Palavras prováveis: bolo, pastel; improvável: batata.
- O modelo codifica o contexto e decodifica a palavra mais provável.

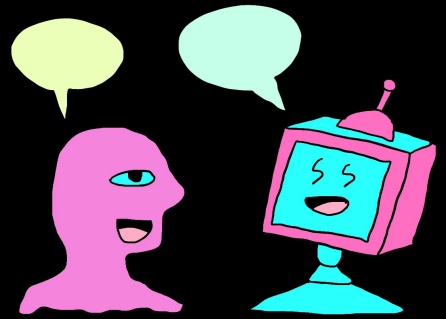


Modelagem de Linguagem – “Preencher a Lacuna” - HARD

- “Fui ao mercado comprar frutas. Comprei banana, maçã e laranja. Quando cheguei em casa, percebi que esqueci de comprar ____.”

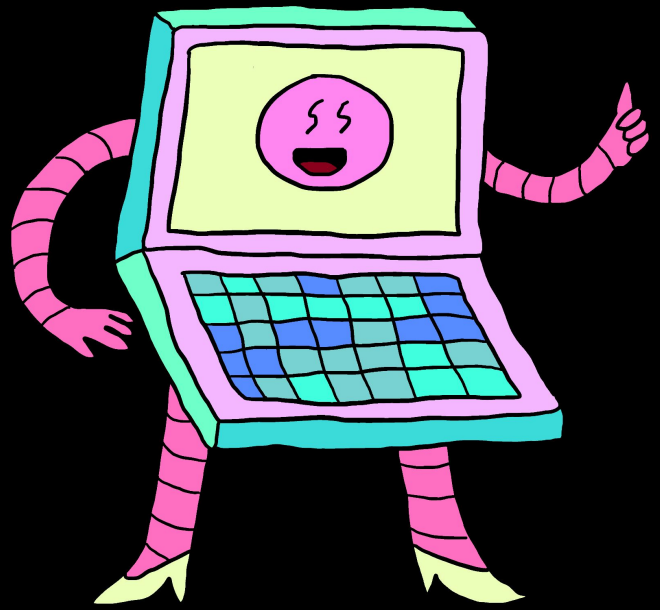
- Palavras prováveis: uva, mamão
- Palavra improvável: pão

- O modelo precisa manter na memória a lista de frutas para sugerir algo coerente.



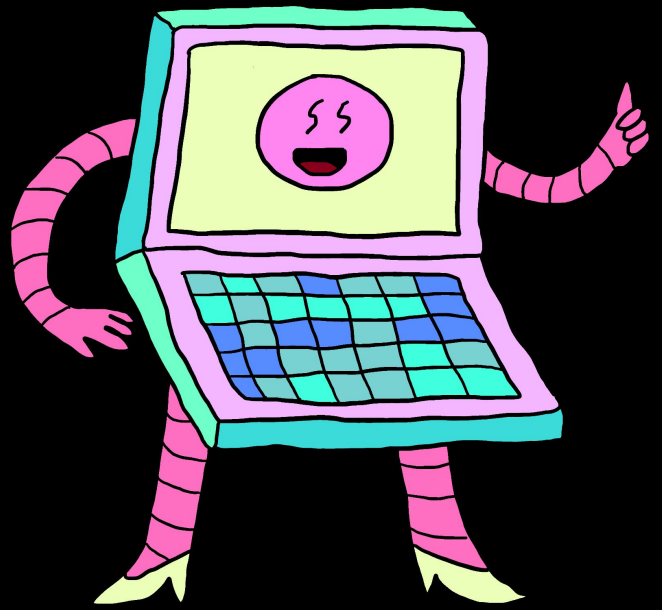
Modelos Encoder-Decoder

Assim como em aprendizado não supervisionado com imagens, podemos representar palavras numericamente. **O modelo observa padrões sozinho, sem receber respostas prontas.**



Modelos Encoder-Decoder

- **Codificador (Encoder):** analisa o texto e decide o que é importante lembrar — captura o significado das palavras e suas relações.
- **Decodificador (Decoder):** usa essas informações para gerar respostas ou tomar decisões, como completar frases ou responder perguntas.

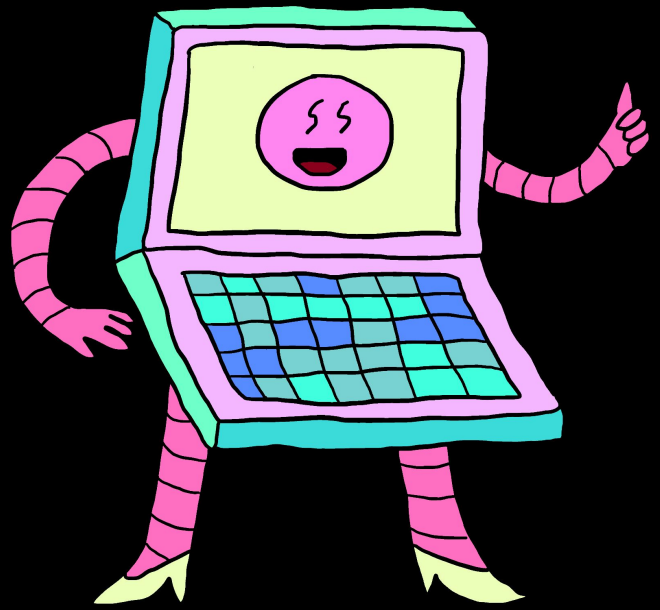


Como implementar Encoder-Decoder na prática?

RNN (Redes Neurais Recorrentes): processam sequências mantendo uma "memória" do que viram antes

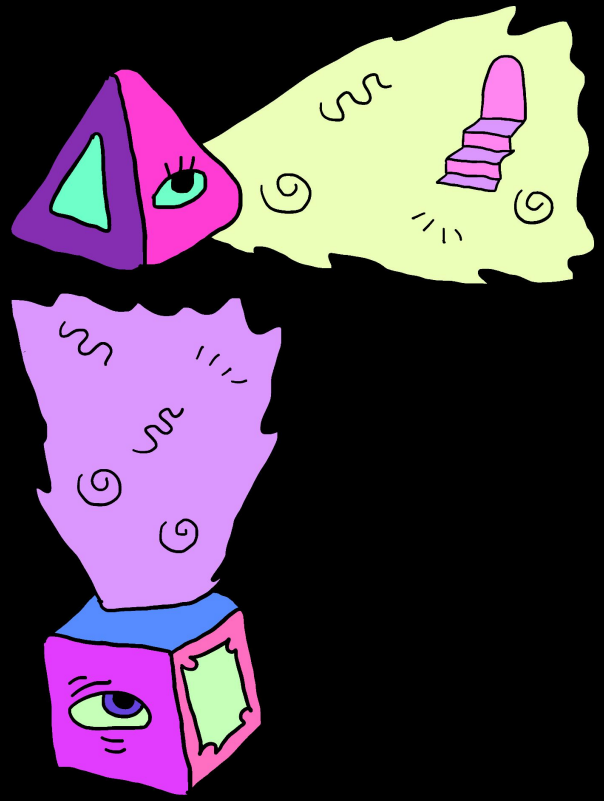
Funcionamento:

- Processam texto palavra por palavra
- Cada passo atualiza um "estado interno"
- O estado carrega informação do contexto anterior.



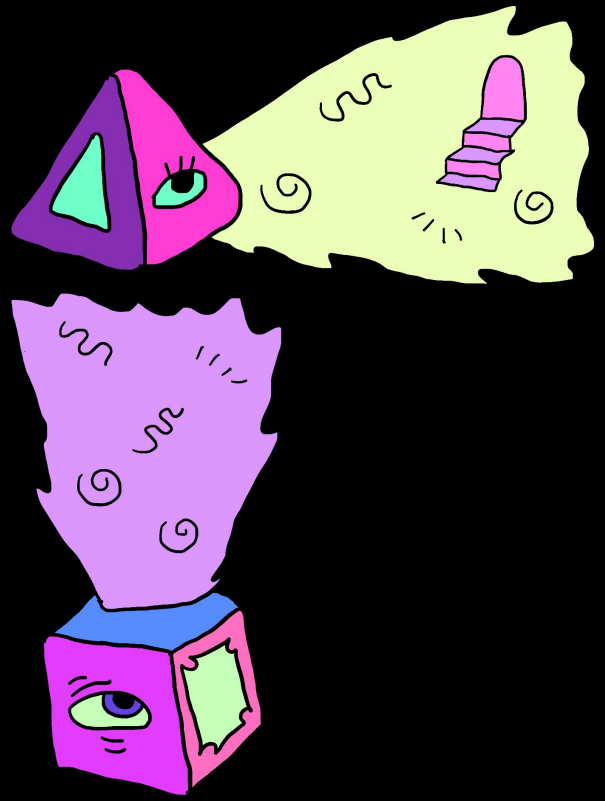
RNN - Processamento sequencial

1. processa → "O"
2. processa → "gato" (lembra de "O")
3. processa → "pulou" (lembra de "O gato")
4. processa → "no" (lembra de "O gato pulou")
5. processa → "sofá" (lembra de "O gato pulou no")



Limitações RNN's

1. Toda a sentença precisa ser comprimida em um único vetor final
2. Difícil lembrar de informações do início em textos longos
3. Processamento lento
4. Informações antigas vão se perdendo ao longo da sequência
5. Como diz Ray Mooney, "*nunca encaixaremos todo o significado de uma sentença em um único vetor*".



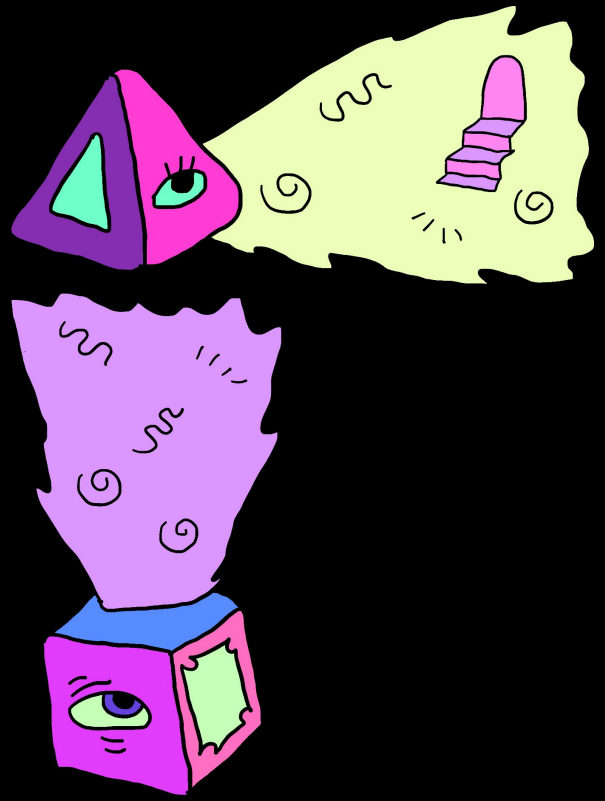
Revolução: Transformers

Ao invés de processar sequencialmente, processa todas as palavras simultaneamente



Processamento simultâneo

1. "O" → olha para → [gato, pulou, no,muro]
2. "gato" → olha para → [O, pulou, no,muro]
3. "pulou" → olha para → [O, gato, no, muro]
4. "no" → olha para → [O, gato, pulou, muro]
5. "muro" → olha para → [O, gato, pulou, no]



```python
from transformers import pipeline

# Use a valid public Portuguese BERT model
fill_mask = pipeline("fill-mask", model="neuralmind/bert-base-portuguese-cased")

resultado = fill_mask("O gato pulou no muro, agora ele está em cima do [MASK] miando.")
for r in resultado:
    print(r["sequence"], "→", round(r["score"], 2))
```



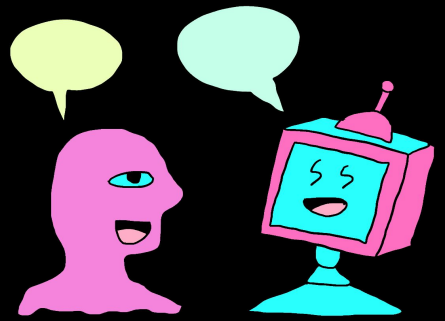
```
O gato pulou no muro, agora ele está em cima do muro miando. → 0.96
O gato pulou no muro, agora ele está em cima do portão miando. → 0.01
O gato pulou no muro, agora ele está em cima do telhado miando. → 0.01
O gato pulou no muro, agora ele está em cima do gato miando. → 0.01
O gato pulou no muro, agora ele está em cima do chão miando. → 0.0
```



Como assim????

Na frase: "A árvore que plantei no jardim cresceu muito"

- RNN: Precisa processar até "jardim" para conectar com "árvore"
- Transformer: "árvore" e "jardim" se conectam diretamente e instantaneamente, independente da distância



Limitações Atuais

- Mesmo transformers têm tamanho máximo de texto que conseguem processar
- Aprendem e replicam preconceitos presentes nos textos de treinamento
- Dificuldade com raciocínios óbvios para humanos
- Podem gerar informações plausíveis mas incorretas
- Custo computacional alto para contextos longos



E o Ray tinha razão?

“Nunca encaixaremos todo o significado de uma sentença em um único vetor”

Ele tinha razão? Em parte...

O que melhoramos?

- **Transformers** não precisam comprimir TUDO em um vetor único
- **Self-attention** permite acessar qualquer palavra diretamente
- **Contexto dinâmico** = significado muda conforme a necessidade



O que persiste?

- Limite prático de contexto (ex.: 4K, 8K tokens)
- Informação ainda se perde em textos muito longos
- Custo computacional cresce exponencialmente

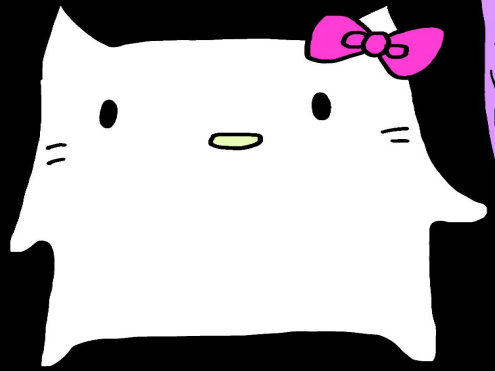


E o Python?

é a linguagem mais usada na área, tanto para pesquisa quanto para aplicações práticas, porque tem uma quantidade enorme de bibliotecas e frameworks prontos que facilitam o trabalho com texto.



Entender como a IA funciona é entender que **nada é mágica** é **tudo matemática, linguagem e intenção**



~~*Thanks for watching*~~

Obrigada por me assistirem!

